Rylan Thomas

Santa Clara University

Electrical and Computer Engineering

ECEN 603 Worksheet 2

1. **Find the product of these two numbers using Booth radix-2 multiplication algorithm**
   -73 and 90 → 8-bits signed

$73 = 01001001 \rightarrow -73 = 10110111$

01011010 $90 = M$

$\times$ 10110111 $-73 = Q$

A = 0000 0000

1 0000 0000 10110111 0 → 10: A − M

00000000 − 01011010 = 10100110

+(10100110)

A Q Q_0

11010011 01011011 1

2 11 → do nothing

A

11101001 10101101 1

3 01 → do nothing

A Q Q_0

11110100 11010110 1

4 [01 → A + M = A

11110100 + 01011010

= 01001110

A Q Q_0

00100111 01101011 0

5 10 → A = A − M

00100111 − 01011010

= 11001101

A Q Q_0

11100110 10110101 1

6 11 do nothing

A Q Q_0

11110011 01011010 1

7 01 → A + M

11110011 + 01011010

= 01001101

A Q Q_0

00100110 10101101 0

8 10 A − M

00100110 − 01011010

= 11001100

A Q Q_0

11100110 01010110 1 → = −6570

2. **Find the product of these two numbers using Booth radix-4 multiplication algorithm**
   -73 and 90

000	nothing
001	+M
010	+M
011	+2M
100	−2M
101	−M
110	−M
111	nothing

A = 0000 0000 0 M = 90 = 01011010 (8-bit) −90 = 10100110 → 9 bit

Q = −73 = 10110111 2M = 10110100

A 10110110 → 110 (−M)

1. 0 0000 0000 + 10100110 = 10100110

A Q Q_0

11110100 10110111 0

2. 011 → +2M

2M = 01011010 0

11110100

+ 01011010 0

= 00100111

A Q Q_0

00100111 01101011 1

3. 110 → −M

00100111

+ 10100110

11001101

A Q Q_0

11110011 01011011 0

4. 101 → −M

11110011

+ 10100110

11001001

A Q Q_0

11110011 01010110 1

1110 0110 0101 0110 = −6570

3. **Describe the Booth radix-4 multiplication algorithm.**

- The radix-4 algo combines 2 radix-2 cycles in one
- It speeds up multiplication by using controlled adders/subtractors in both series and parallel (ripple-style)
- You start w/ the 2 numbers Q and M. Keep M the same always. Have a reg A that is the size of largest(Q,M) +1. Set it to 0. Place as such:

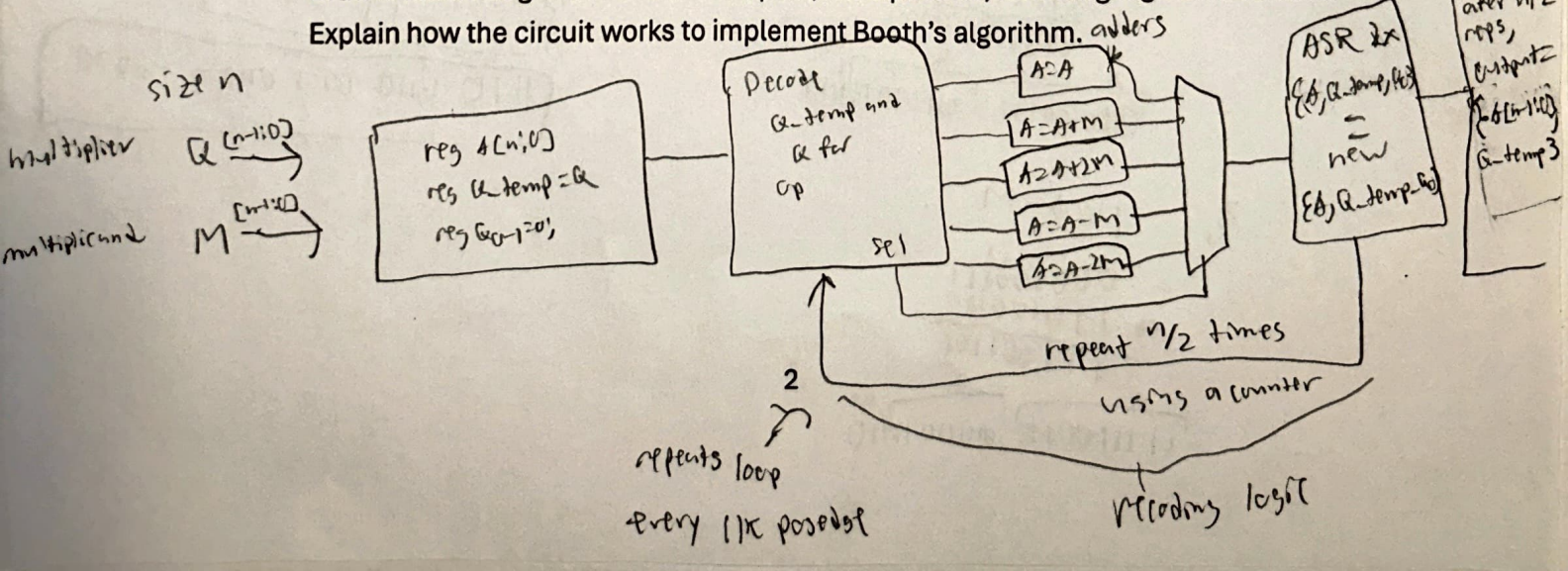
A Q Q_{0-1} ← default start at 0

depending on Q_1, Q_0, and Q_{0-1}: refer to the truth table. After computing, ASR twice (discard last two bits) and find your new A, Q, Q_0. Repeat until you've finished n/2 cycles (n = largest(Q,M))

truth table

Q_1	Q_0	Q_{0-1}	
0	0	0	A=A
0	0	1	A=A+M
0	1	0	A=A+M
0	1	1	A=A+2M
1	0	0	A=A-2M
1	0	1	A=A-M
1	1	0	A=A-M
1	1	1	A=A

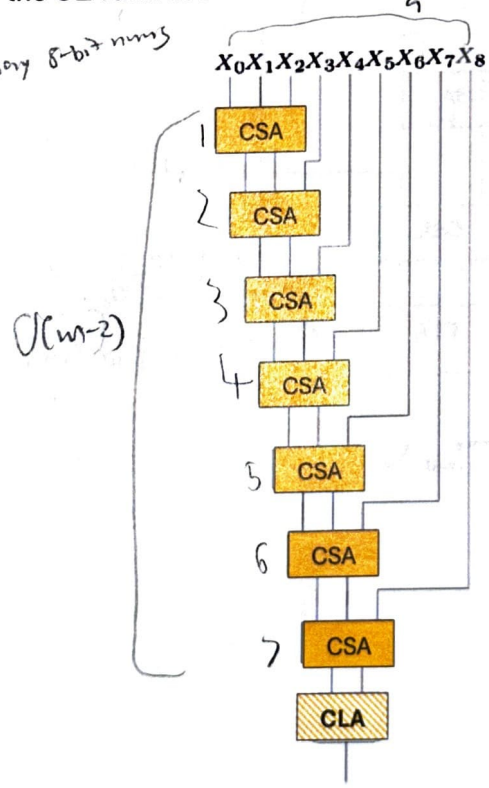
4. **In the lecture, we discussed a combinational array-based design for Booth's radix-4 multiplication. Modify this design to implement a *synchronous* circuit for Booth's radix-4 multiplication algorithm. The 8-bit multiplicand should be stored in a register and the multiplier bits should be shifted in two bits at a time into the multiplier. Draw a block diagram with the multiplier, multiplicand, recoding logic and adders. Explain how the circuit works to implement Booth's algorithm.**



5. (a) The figure below shows nine 8-bit numbers are added together using a *chain* of carry-save adder (CSA) blocks. The first CSA block adds 3 numbers and each subsequent CSA adds a new number to the sum and carry bits from the previous CSA block. Note that each CSA block contains several 1-bit carry-save adders in parallel. (For example, problem 2 of Worksheet 1 has a chain of 2 CSA blocks.) In the last stage, a carry lookahead adder (CLA) is used. What is the total gate delay of a design that uses this chain structure of CSAs to add *m* different *n*-bit numbers together? Give your answer in terms of *m* and *n* using asymptotic notation.
(Hint: A single CSA block takes only O(1) time. Calculate how many levels of CSA blocks are needed to find delay through CSA blocks. Determine the total size of the input to the CLA and use this information to calculate CLA delay.)

(let's say 8-bit nums

m diff n bits nums



$O(m-2)$

total $= O((m-2) + \log_2(n + \log_2(m)))$

$\approx O(m + \log(n + \log(m)))$

num size 2

$9(2^8-1) =$

2295

how many bits

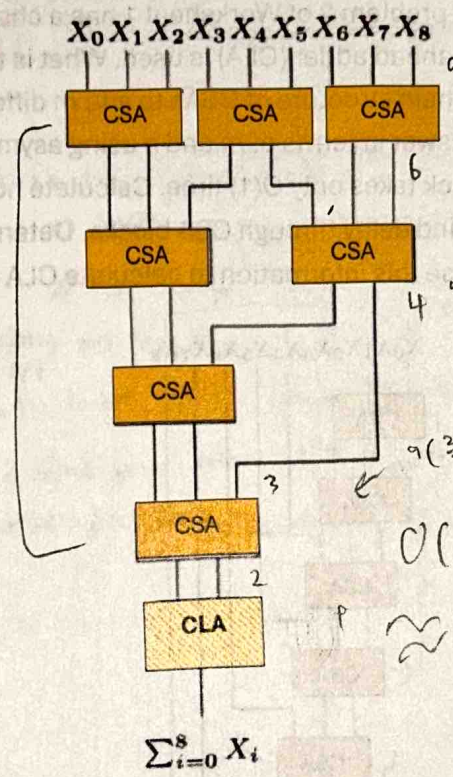
$\log_2(2295) \approx 11.164 = 8 + \log_2(9)$

$n + \log_2(m)$ bits

CLA delay $= \log_2$(bit width)

$O(\log(n + \log(m)))$

(b) Instead of using a chain of CSA blocks, a tree formation can be used as shown in the figure below. This structure is known as Wallace tree. What is the gate delay when a tree of CSAs is used to add *m* different *n*-bit numbers together? Give your answer in terms of *m* and *n* using asymptotic notation.



$m=9$

$9\ (0)\ \left(\frac{2}{3}\right)^0$

$9\left(\frac{2}{3}\right)\ \left(\frac{2}{3}\right)^1$

$9\left(\frac{4}{9}\right)\ \left(\frac{2}{3}\right)^2$

now $\log_{3/2}(m-2)$

logarithmic, but not exactly $\log_{3/2}$

$9\left(\frac{2}{3}\right)^3 \neq 3$ tno

$O(\log_{3/2}(m-2) + \log_2(n+\log_2(m)))$

$\approx O(\log(m) + \log(n+\log(m)))$